

Tensors trong PyTorch

```
1 import torch
2
3 # Khoi tao
4 x = torch.ones(size=(2, 1))
5 y = torch.zeros(size=(2, 1))
6 z = torch.tensor([1, 2, 3], dtype=torch.float32)
7
8 # Random (cu phap hoi khac biet)
9 r1 = torch.normal(mean=torch.zeros((3,1)), std=torch.ones((3,1)))
10 r2 = torch.rand(3, 1) # Uniform
11
```

Gán Tensor và nn.Parameter

Khác với TF, PyTorch Tensors có thể trực tiếp gán giá trị:

```
1 x = torch.zeros(size=(2, 1))  
2 x[0, 0] = 1.  
3
```

Nhưng để theo dõi đạo hàm, ta sử dụng thuộc tính `requires_grad=True`:

```
1 v = torch.ones(size=(3, 1), requires_grad=True)  
2 # Hoac su dung Parameter khi viet Module  
3 import torch.nn as nn  
4 w = nn.Parameter(torch.rand(3, 3))  
5
```

Tính toán Gradient bằng .backward()

Cú pháp đạo hàm của PyTorch đơn giản hơn (không cần context manager GradientTape):

```
1 x = torch.tensor(3.0, requires_grad=True)
2 result = torch.square(x)
3
4 # Goi backward de tinh dao ham cho toan bo graph
5 result.backward()
6
7 # Xem dao ham cua result theo x
8 print(x.grad) # 6.0
9
```

Tổ chức code với nn.Module

```
1 class SimpleDense(nn.Module):
2     def __init__(self, in_channels, out_channels):
3         super().__init__()
4         self.W = nn.Parameter(torch.rand(in_channels, out_channels))
5         self.b = nn.Parameter(torch.zeros(out_channels))
6
7     def forward(self, x):
8         return torch.relu(torch.matmul(x, self.W) + self.b)
9
10 model = SimpleDense(64, 32)
11
```

- $JAX = Numpy + Autograd + XLA$ (Trình biên dịch tăng tốc cực độ).
- Phổ biến ở Google DeepMind, Midjourney, Anthropic do khả năng chạy cực lớn trên cụm TPU.
- **Lập trình hàm không trạng thái (Stateless):** Mọi hàm đều trong suốt (pure functions), không sử dụng biến toàn cục. Giúp JAX an toàn khi chạy song song.

jax.numpy và PRNG Keys

JAX tái tạo 99% API của Numpy thông qua `jax.numpy`:

```
1 from jax import numpy as jnp
2 x = jnp.ones(shape=(2, 1))
3
```

PRNG Keys (Số ngẫu nhiên không trạng thái):

```
1 import jax
2 # Phai tao "chia khoa" (key) de random
3 seed_key = jax.random.key(1337)
4 r1 = jax.random.normal(seed_key, shape=(3,))
5
6 # De lay random tiep theo, phai tao key moi
7 new_key = jax.random.split(seed_key, num=1)[0]
8
```

Gán Tensor trong JAX

Giống TensorFlow, JAX tensor không thể gán trực tiếp. Phải dùng cú pháp `.at[] .set()` để trả về một tensor mới:

```
1 x = jnp.array([1, 2, 3], dtype="float32")
2 # x[0] = 10 se bao loi
3 new_x = x.at[0].set(10)
4
```

Tính toán Gradient bằng jax.grad()

JAX sử dụng phương pháp "siêu lập trình" (hàm trả về hàm) để tính đạo hàm:

```
1 def compute_loss(input_var):  
2     return jnp.square(input_var)  
3  
4 # Truyền hàm loss qua jax.grad để lấy hàm tính đạo hàm  
5 grad_fn = jax.grad(compute_loss)  
6  
7 x = jnp.array(3.0)  
8 print(grad_fn(x)) # 6.0  
9
```


Trả về cả Loss và Gradient

Thực tế, ta thường cần cả giá trị Loss và cấu trúc Gradient. JAX cung cấp `value_and_grad()`:

```
1 def compute_loss(state, x, y):  
2     # state có thể là tuple (W, b)  
3     pass  
4  
5 # Trả về một hàm mới  
6 grad_fn = jax.value_and_grad(compute_loss)  
7  
8 # Tính toán đồng thời  
9 loss, grads = grad_fn(state, x, y)  
10
```

Biên dịch hàm cực nhanh với @jax.jit

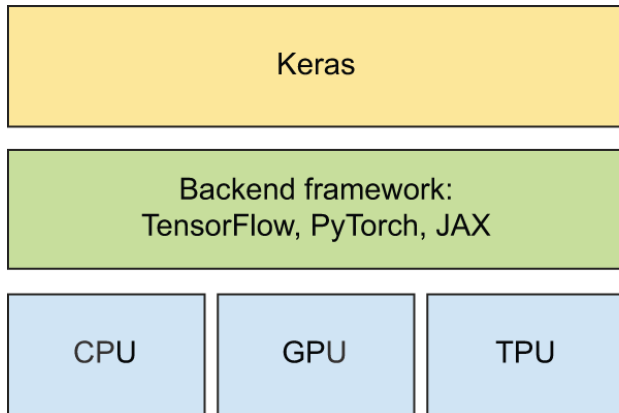
```
1 @jax.jit
2 def dense(inputs, W, b):
3     return jax.nn.relu(jnp.matmul(inputs, W) + b)
4
```

- JAX biên dịch hàm này thành mã máy XLA.
- *Ràng buộc*: Hàm phải phi trạng thái (không sửa đổi biến ngoài, cập nhật tensor phải trả về).

Keras: High-Level API cho Deep Learning

- Keras ra đời (2015) để giúp lập trình Deep Learning giống như chơi "lắp ghép LEGO".
- Keras che đi toàn bộ sự phức tạp của Low-level API (viết Gradient, vòng lặp lặp).
- Được sử dụng bởi Google, Netflix, Uber, Waymo.

Kiến trúc đa Backend của Keras



Deep learning development:
layers, models, optimizers, losses,
metrics, training loops ...

Tensor manipulation infrastructure:
tensors, variables, automatic
differentiation, distribution ...

Hardware: execution

Hình 3.1: Cấu trúc đa Backend của Keras 3.0

Layer cơ sở: Thiết kế theo hướng Đối tượng

Trong Keras, Layer là khối xây dựng cơ bản, gồm trạng thái (weights) và tính toán (forward pass).

```
1 import keras
2 class SimpleDense(keras.Layer):
3     def __init__(self, units):
4         super().__init__()
5         self.units = units
6
7     def build(self, input_shape): # Goi 1 lan duy nhat
8         self.W = self.add_weight(shape=(input_shape[-1], self.units))
9         self.b = self.add_weight(shape=(self.units,))
10
11    def call(self, inputs): # Thuc thi tinh toan
12        return keras.ops.matmul(inputs, self.W) + self.b
13
```

Lợi ích của hàm build()

Hàm build() cho phép Keras **tự động suy diễn hình dạng (shape inference)**. Bạn không cần chỉ định đầu vào dài bao nhiêu:

```
1 # Bạn không cần quan tâm lớp trước truyền vào bao nhiêu unit
2 model = keras.Sequential([
3     SimpleDense(32), # Tu dong hieu input_shape o day la 784
4     SimpleDense(64),
5     SimpleDense(10)
6 ])
7 # Goi model tren du lieu thuc te se kích hoạt build() hàng loạt
8
```

Biên dịch Mô hình (model.compile)

Để cấu hình quá trình học tập, Keras có bước compile cực kỳ chuẩn hóa:

```
1 model.compile(  
2     optimizer="adam",      # Phep toi uu gradient descent  
3     loss="mse",           # Ham mat mat  
4     metrics=["accuracy"]  # Do do hieu suat hien thi  
5 )  
6 # Train tren du lieu  
7 model.fit(x_train, y_train, epochs=10)  
8
```